# Abstract

## Abstract I

Research software repositories in monorepo configurations accumulate three categories of shared resources that individual projects must consume without re-implementing discovery logic: **data pools** (bibliographies, contacts, datasets), **governance rules** (style guides, coverage thresholds, citation schemas), and **executable tools** (code executors, validators, skill invocations). Without a canonical integration pattern, projects either duplicate discovery logic or silently ignore resources that fail to load — both outcomes degrade reproducibility and collaborative cohesion (Wilson et al. 2014; Taschuk and Wilson 2017).

## Abstract I

This paper presents `template_pools_rules_tools`, a meta-project exemplar that demonstrates how a single project can programmatically discover, validate, and exercise all three resource categories with zero tight coupling to any specific resource instance. The exemplar comprises 13 Python modules, including resource readers, an integration orchestrator, semantic rule evaluation, figure generation, manuscript-token generation, resource-schema validation, and shared type definitions, plus 6 thin orchestration scripts and a fully token-injected manuscript pipeline.

## Abstract I

The architecture (fig. 1) separates *resource ownership* from *resource consumption*. Resources live in top-level `fonds/`, `rules/`, and `tools/` directories and are never modified by consumers. Each resource exposes a typed manifest (`fonds.yaml`, `rules.yaml`, `tools.yaml`) that the corresponding reader module uses for discovery and validation. All readers implement graceful fallbacks: they return `None` or empty collections when a resource is absent, log a warning via the standard library `logging` module, and allow the integration pipeline to continue. This revision extends the original three-figure presentation to 8 content figures plus 1 cover illustration — a fond taxonomy (fig. 2), a rule hierarchy (fig. 3), a tool invocation contract (fig. 4), a three-level resilience diagram (fig. 8), and a script pipeline flow (fig. 6) — so that every structural claim in

the prose has a corresponding visual.

In a representative pipeline run, the integration demo loaded 3 fonds, validated 2 rule sets, discovered 4 tools, and processed 8 bibliography entries — all reported as structured JSON that populates manuscript variable tokens at render time. Tests covering the 13 `src/` modules across 13 test files achieve above the required $\geq$90% combined line coverage and use real file paths rather than mocks; rerun the project pytest command for current measured coverage.

# Abstract I

The `template_pools_rules_tools` exemplar provides a reference implementation that any project in the template repository can consult when designing its own resource-consumption layer.

This reference implementation is deliberately reproducible end-to-end: every count, figure, and page-layout choice in this document is regenerated from source rather than hand-authored, per the Reproducibility Checklist in the front matter.

**Keywords:** research software engineering, monorepo architecture, reproducibility, fonds, governance rules, tool discovery, graceful degradation